



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

REV	DATA	ZMIANY
0.1	11.06.2026	Maciej Skrobisz

Raport projektu

Multisport tracker TinyML - urządzenie wearable do śledzenia ćwiczeń

Autor: Maciej Skrobisz

Przedmiot: Metodyki projektowania i modelowania systemów 1

Spis treści

1. Cel projektu	3
2. Analiza i dobór platformy sprzętowej.....	4
3. Iteracyjny proces projektowania zasilania i obudowy	5
4. Środowisko programistyczne i architektura oprogramowania	8
4.1. Wykorzystane wzorce projektowe (Design Patterns)	8
4.2. Modelowanie systemu (UML).....	9
5. Algorytmy inferencji i przetwarzania danych w czasie rzeczywistym.....	10
5.1. Logika Przesuwnego Okna (Sliding Window)	10
5.2. Histereza i Cyfrowy "Debouncing" (Eliminacja drgań styków).....	10
5.3. Dwukierunkowa Komunikacja (Bluetooth Low Energy).....	11
6. Projektowanie i implementacja systemu uczenia maszynowego (Edge AI)	12
6.1. Akwizycja i struktura zbioru danych (Dataset).....	12
6.2. Konfiguracja okna czasowego (Time Series Configuration).....	13
6.3. Analiza spektralna i cyfrowe przetwarzanie sygnałów (DSP).....	14
6.4. Architektura i uczenie sieci neuronowej	14
7. Opis wykonanych testów.....	16
7.1. Ewaluacja statystyczna modelu (Edge Impulse)	16
7.2. Walidacja empiryczna i przypadki brzegowe (Edge Cases).....	17
7.3. Rejestr usterek i modyfikacji kodu.....	18
8. Podręcznik użytkownika	21
8.1. Konfiguracja środowiska deweloperskiego	21
8.2. Prototypowanie i budowanie projektu (Build Configuration).....	21
8.3. Procedura wgrywania oprogramowania (Flashing via Terminal).....	21
9. Metodologia rozwoju i utrzymania systemu.....	23
9.1. Ścieżki optymalizacji algorytmów TinyML	23
9.2. Projektowanie mechaniczne i konserwacja sprzętu	23
9.3. Zarządzanie cyklem życia oprogramowania i aktualizacje	24

1. Cel projektu

Głównym celem projektu było zaprojektowanie i wykonanie wbudowanego, przenośnego systemu (wearable) zdolnego do autonomicznego rozpoznawania i zliczania powtórzeń ćwiczeń fizycznych w czasie rzeczywistym. Założono, że urządzenie będzie montowane na ramieniu użytkownika za pomocą opaski, a proces inferencji sztucznej inteligencji (Edge AI) będzie odbywał się w pełni lokalnie na mikrokontrolerze, bez konieczności ciągłego przesyłania surowych danych do jednostki nadrzędnej.

Zadaniem urządzenia jest odciążenie użytkownika od manualnego monitorowania treningu. Po przetworzeniu danych z wbudowanego akcelerometru i sklasyfikowaniu ćwiczenia, system przesyła jedynie zwięzłe podsumowania (np. zaliczone powtórzenie lub czas trwania aktywności) do smartfona z wykorzystaniem energooszczędnego protokołu Bluetooth Low Energy (BLE).

2. Analiza i dobór platformy sprzętowej

Przy doborze sprzętu najważniejsze były niewielkie rozmiary, niski pobór energii oraz wystarczająca moc obliczeniowa. Integracja akcelerometru (IMU) bezpośrednio na mikrokontrolerze (SoC) była priorytetem, ponieważ oddzielne moduły zwiększałyby objętość obwodu, wprowadzając jednocześnie podatność na uszkodzenia mechaniczne w miejscu połączeń.

W procesie decyzyjnym rozpatrzono trzy architektury:

Platforma	Zalety	Wady
Opcja 1: ESP32 + zewnętrzny IMU (np. ICM- 20948)	Duża moc obliczeniowa, ogromne wsparcie społeczności, brak potrzeby zakupu komponentów.	Duże rozmiary elementów, wykorzystanie zawodnych połączeń przewodowych. Wysoki pobór prądu przez układ ESP32.
Opcja 2: Arduino Nano 33 BLE Sense	Branżowy standard do prototypowania TinyML, wbudowane czujniki.	Stosunkowo duże gabaryty fizyczne (45x18 mm). Starsza architektura rdzenia (Cortex-M4). Wysoka cena modułu.
Opcja 3: Seeed Studio XIAO nRF54L15 Sense	Świetna miniaturyzacja (21x17.5 mm). Szybki, energooszczędny procesor Nordic Cortex-M33 (128 MHz). Zintegrowany, precyzyjny układ IMU.	Nowa platforma na rynku (mniejsza ilość gotowych bibliotek), wyższy próg wejścia w środowisko nRF Connect SDK.

Wybór ostateczny: Wybrano platformę **XIAO nRF54L15 Sense**. Rozwiązanie to przewyższa pozostałe ze względu na stosunek rozmiaru do wydajności.

Dodatkowym czynnikiem decydującym była chęć implementacji projektu na architekturze Nordic Semiconductor, często stosowanej w rozwiązaniach wymagających BLE o niskim poborze mocy. Warstwę programistyczną i trening

modelu zrealizowano z wykorzystaniem systemu Zephyr RTOS oraz platformy Edge Impulse.

3. Iteracyjny proces projektowania zasilania i obudowy

Konstrukcja warstwy zasilającej oraz ergonomia urządzenia wymagały przeprowadzenia trzech iteracji projektowych ze względu na ograniczenia fizyczne oraz sprzętowe awarie komponentów.

- **Iteracja 1: Bezpośrednie lutowanie ogniwa Li-Po**



Rys. 1 Akumulator Li-Po Akyga LP301525 (3.7V, 85mAh)

Wstępne założenia obejmowały bezpośrednie przylutowanie akumulatora Li-Po Akyga LP301525 (3.7V, 85mAh) do dolnych padów (BAT/GND) mikrokontrolera XIAO. Podejście to zakończyło się krytyczną awarią sprzętową. Analiza wykazała, że najprawdopodobniej układ zarządzania energią (PMIC) na mikrokontrolerze uległ uszkodzeniu. Awaria ta wymusiła zwrot uszkodzonego komponentu i zmianę podejścia na wariant nieingerujący w strukturę lutowniczą tak delikatnych układów mocy.

- **Iteracja 2: Płytki rozszerzeń Expansion Board**



Rys. 2 Seeeduino XIAO Expansion Board

W drugiej fazie wykorzystano dedykowaną płytkę Seeeduino XIAO Expansion Board z większym akumulatorem. Rozwiązanie to okazało się niepraktyczne w docelowym scenariuszu użycia. Duży, prostokątny format płytki powodował znaczny dyskomfort pod opaską naramienną, a wbudowane w płytkę peryferia (np. ekran OLED) generowały wysoki, pasożytniczy pobór prądu, znacznie skracając czas pracy urządzenia.

- **Iteracja 3: Architektura finalna**

Ostatecznie zrezygnowano ze zintegrowanych pakietów Li-Po na rzecz zewnętrznego zasilania. Zastosowano smukły powerbank o wymiarach 9x2x2 cm (kształt podłużny). Płytkę XIAO nRF54L15 została umieszczona w miniaturowej, przezroczystej obudowie z plastiku i usztywniona pianką w celu tłumienia drgań. Taka konstrukcja zapewniła izolację układu od czynników zewnętrznych takich jak pot i niechciane wibracje oraz ergonomiczny, obły kształt, dobrze dopasowujący się do ramienia użytkownika.



Rys. 3 XIAO nRF54L15 w plastikowej obudowie



Rys. 4 XIAO nRF54L15 oraz powerbank w opasce do biegania na ramię

4. Środowisko programistyczne i architektura oprogramowania

Oprogramowanie układowe (firmware) zostało zrealizowane w oparciu o system czasu rzeczywistego Zephyr RTOS, stanowiący znakomite rozwiązanie dla układów o niskim poborze mocy. Proces kompilacji i zarządzania zależnościami przeprowadzono z wykorzystaniem oficjalnego środowiska nRF Connect SDK (wersja 3.0.1) oraz rozszerzenia nRF Connect for VS Code.

Punktem wyjścia dla warstwy pobierania danych z akcelerometru był kod referencyjny udostępniony przez producenta (Seeed Studio). Został on jednak zmodyfikowany i rozszerzony, aby obsłużyć wielowątkowość Zephyra, stos komunikacyjny Bluetooth Low Energy oraz zintegrowane biblioteki sztucznej inteligencji wygenerowane przez platformę Edge Impulse.

4.1. Wykorzystane wzorce projektowe (Design Patterns)

Aby zapewnić czytelność, skalowalność i niezawodność kodu, architekturę systemu oparto na sprawdzonych wzorcach projektowych z inżynierii oprogramowania:

1. **Wzorzec Maszyny Stanowej (State Pattern):** Główny algorytm zliczający ćwiczenia opiera się na ciągłym przechodzeniu układu między zdefiniowanymi stanami:

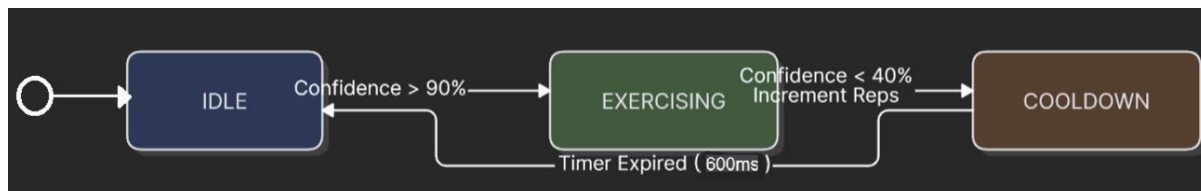
Oczekiwanie (idle) -> Wykryto Ruch (is_exercising = true) -> Zakończono Powtórzenie (zaliczenie) -> Czas Stygnięcia (cooldown).

Zapobiega to nakładaniu się na siebie logiki różnych ćwiczeń.

2. **Potoki i Filtry (Pipes and Filters):** Wzorzec architektoniczny zastosowany do przetwarzania sygnału (Digital Signal Processing). Surowe dane z 3 osi akcelerometru wpływają do systemu jako strumień (Potok), po czym są kolejno modyfikowane: *Bufor okna przesuwanego -> Skalowanie -> Transformata Fouriera (FFT) -> Warstwy Sieci Neuronowej.*
3. **Wzorzec Obserwatora (Observer Pattern / Callbacks):** Wykorzystany w asynchronicznej obsłudze stosu Bluetooth. Mikrokontroler nie odpytuje ciągle interfejsu radiowego. Zamiast tego rejestruje funkcje zwrotne (tzw. Callbacks, np. `bt_receive_cb`), które są automatycznie wywoływane przez system operacyjny dopiero w momencie, gdy z telefonu nadejdzie zewnętrzny bodziec (komenda).

4.2. Modelowanie systemu (UML)

W celu zwizualizowania logiki decyzyjnej zaimplementowanej w pętli głównej (main.cpp) wykorzystano diagram stanów UML. Obrazuje on cykl życia pojedynczego powtórzenia ćwiczenia oraz mechanizmy filtrujące błędne odczyty.



Rys. 5 Diagram UML maszyny stanów zaprogramowanej w projekcie

5. Algorytmy inferencji i przetwarzania danych w czasie rzeczywistym

Przeniesienie obliczeń z chmury bezpośrednio na mikrokontroler (Edge Computing) wymagało implementacji dedykowanych algorytmów optymalizujących pamięć i czas procesora.

5.1. Logika Przesuwnego Okna (Sliding Window)

Próbkowanie z akcelerometru odbywa się z częstotliwością 50 Hz. Model sztucznej inteligencji wymaga na wejściu 3-sekundowego "okna" danych (łącznie ok. 450 wartości typu float). Aby zachować płynność działania i zminimalizować opóźnienia (Latency), zaimplementowano algorytm przesuwnego okna. Zamiast czekać 3 sekundy na zebranie nowych danych do każdej predykcji, bufor przesuwany jest jedynie o ułamek czasu (tzw. krok / Stride = 200 ms). Najstarsze próbki są usuwane za pomocą funkcji *memmove()*, robiąc miejsce na nowe, co pozwala na generowanie wyniku inferencji 5 razy na sekundę przy minimalnym zużyciu RAM-u.

5.2. Histereza i Cyfrowy "Debouncing" (Eliminacja drgań styków)

Analiza testów empirycznych wykazała problem "podwójnego zliczania" powtórzeń w przypadku bardzo wolnego wykonywania ćwiczenia (np. zatrzymanie ruchu w połowie pompki powodowało fałszywy spadek pewności predykcji). W celu zniwelowania tego zjawiska wprowadzono dwa algorytmy ochronne:

- **Histereza detekcji:** Rozpoczęcie ćwiczenia wymaga od sieci neuronowej wysokiej pewności klasyfikacji na poziomie 90% (*START_THRESHOLD* = 0.90). Jednakże, aby uznać ruch za zakończony, pewność modelu musi spaść poniżej 40% (*END_THRESHOLD* = 0.40).
- **Mechanizm Cooldown (Czas stygnięcia):** Po każdym zaliczonym powtórzeniu algorytm nakłada na system blokadę trwającą 3 okna = 600 ms. W tym czasie piki pewności modelu są ignorowane, co działa analogicznie do sprzętowego filtrowania drgań styków (debouncing) w klasycznej elektronice cyfrowej. Zabezpiecza to przed zliczeniem końcowej fazy jednego ruchu jako początku drugiego.

5.3. Dwukierunkowa Komunikacja (Bluetooth Low Energy)

Do komunikacji z telefonem wykorzystano aplikację Serial Bluetooth Terminal by Kai Morich. Urządzenie wykorzystuje protokół Nordic UART Service (NUS) do emulowania bezprzewodowego portu szeregowego. Oprócz asynchronicznego wysyłania powiadomień o zaliczonych ćwiczeniach, system obsługuje komendy przychodzące ze smartfona:

- **Żądanie Resetu (0x31 / '1')**: Wywołuje przerwanie, które natychmiastowo zeruje wszystkie wskaźniki tablicy *rep_counts* i liczniki czasu, przygotowując urządzenie do nowej sesji treningowej.
- **Żądanie Podsumowania (0x32 / '2')**: Generuje zbiór pakietów z iteracyjnym opóźnieniem *k_msleep(50)*. Opóźnienie to jest kluczowe z punktu widzenia specyfikacji stosu BLE, ponieważ zapobiega przepełnieniu bufora MTU (Maximum Transmission Unit) i gwarantuje bezstratne dostarczenie kompletnej listy wykonanych ćwiczeń do jednostki nadrzędnej.

```
13:24:08.664 brzuszki: 6
13:24:18.921 2
13:24:18.972
13:24:18.972 #####
13:24:19.031 brzuszki: 6
13:24:19.061 pajacyki: 2
13:24:19.121 pompki: 6
13:24:19.181 bieg: 0 s
13:24:19.181 #####
13:24:21.190 1
13:24:21.250
13:24:21.250 [ ZRESETOWANO LICZNIKI ]
13:24:37.401 brzuszki: 1
13:24:39.701 brzuszki: 2
```

Rys. 6 Dwukierunkowa komunikacja w Serial Bluetooth Terminal

6. Projektowanie i implementacja systemu uczenia maszynowego (Edge AI)

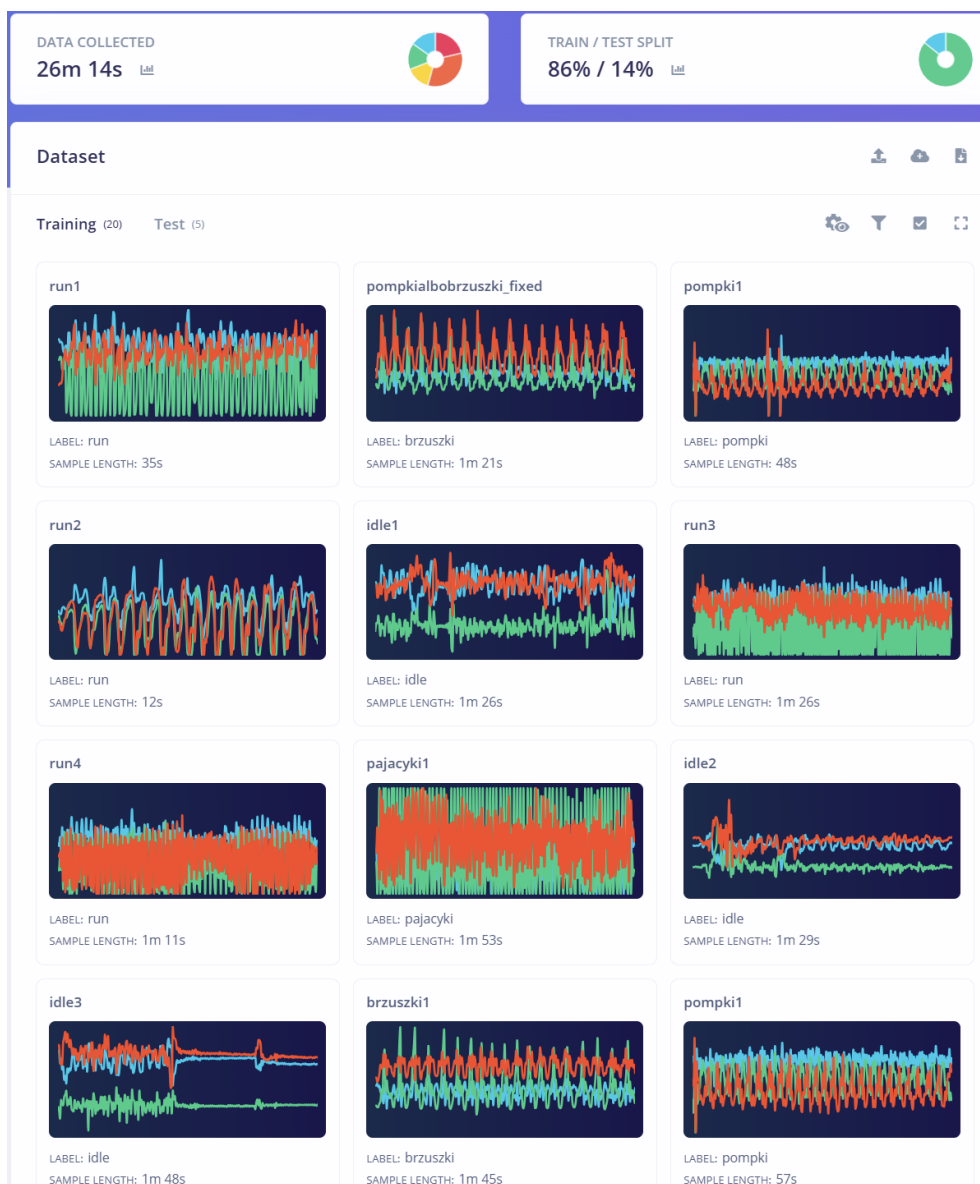
Wytworzenie stabilnego klasyfikatora ruchu wymagało przejścia przez pełen cykl metodyki **CRISP-DM** (Cross-Industry Standard Process for Data Mining), dostosowanej do ograniczeń systemów wbudowanych. Zamiast klasycznych algorytmów progowych (które zawodzą przy wieloosiowych, nieliniowych składowych ruchu ludzkiego), zdecydowano się na zastosowanie sztucznej sieci neuronowej (ANN) współpracującej z cyfrowym przetwarzaniem sygnału (DSP).

6.1. Akwizycja i struktura zbioru danych (Dataset)

Dane treningowe zostały zebrane bezpośrednio za pomocą sensorów IMU platformy XIAO. Łączny czas nagrań wyniósł 26 minut i 14 sekund. Zbiór danych został podzielony na 5 dyskretnych klas reprezentujących docelowe aktywności oraz stan spoczynku:

- **brzuszki**
- **pompki**
- **pajacyki**
- **run** (bieg/trucht w miejscu)
- **idle** (stan spoczynkowy, brak celowej aktywności fizycznej)

W celu obiektywnej ewaluacji zdolności generalizacyjnych modelu, zbiór danych został trwale podzielony na podzbiór uczący (**Training set – 86%**) oraz podzbiór testowy (**Test set – 14%**). Dane testowe zostały całkowicie odizolowane od procesu optymalizacji wag sieci.



Rys. 7 Wizualizacja zebranych danych na platformie Edge Impulse

6.2. Konfiguracja okna czasowego (Time Series Configuration)

Przetwarzanie sygnału ciągłego z akcelerometru (próbkowanego z częstotliwością **50 Hz**) wymagało zaimplementowania podziału na ramki czasowe o następujących parametrach:

- **Rozmiar okna (Window size):** 3000 ms (3 sekundy)
- **Krok okna (Window stride):** 200 ms

Wybór 3-sekundowego okna wynika bezpośrednio z biomechaniki ludzkiego ciała. Pełen cykl anatomiczny jednego poprawnego powtórzenia (np. faza ekscentryczna i koncentryczna pompki lub brzuszka) trwa średnio od 1 do 2.5 sekundy. Okno o długości 3 sekund gwarantuje, że wewnątrz przetwarzanej ramki zawsze znajdzie

się co najmniej jeden pełny, nieprzerwany cykl ruchu, co jest warunkiem koniecznym do poprawnej ekstrakcji cech częstotliwościowych.

- **Konsekwencje zbyt krótkiego okna (np. 1s):** Model traci kontekst cyklicznego ruchu. Widzi jedynie odizolowany fragment (np. sam ruch w dół), co uniemożliwia odróżnienie pompki od nagłego opuszczenia ramienia, drastycznie obniżając skuteczność klasyfikacji.
- **Konsekwencje zbyt długiego okna (np. 6s):** Powoduje drastyczny wzrost zużycia pamięci RAM na mikrokontrolerze oraz wprowadza niedopuszczalne opóźnienie (latency) w systemie czasu rzeczywistego.

Krok okna wynoszący 200 ms zapewnia nakładanie się ramek (overlap) na poziomie ok. 93%. Pozwala to na uruchomienie inferencji 5 razy na sekundę, co zapewnia natychmiastową reakcję interfejsu Bluetooth na ruch użytkownika.

6.3. Analiza spektralna i cyfrowe przetwarzanie sygnałów (DSP)

Zanim surowy wektor przyspieszenia [X, Y, Z] trafi do sieci neuronowej, poddawany jest filtracji i transformacji dziedzinowej w bloku DSP (Spectral Analysis):

1. **Filtr dolnoprzepustowy (Low-pass filter):** Zastosowano cyfrowy filtr 8. rzędu (8th-order) o częstotliwości odcięcia (Cut-off) **10 Hz**. Ruchy człowieka charakteryzują się niską częstotliwością (poniżej 5 Hz). Wyższe składowe harmoniczne w sygnale akcelerometru to szum tła, drżenia mięśniowe lub mikro-wstrząsy obudowy tracker'a. Filtr ten skutecznie eliminuje te zakłócenia.
2. **Dyskretna Transformata Fouriera (FFT):** Wykorzystano algorytm FFT o długości **128 punktów**. Przekształca on przefiltrowany sygnał z dziedziny czasu do dziedziny częstotliwości. Cechy częstotliwościowe wyznaczone na podstawie FFT stanowią unikalny "odcisk palca" dla każdego typu ćwiczenia.

6.4. Architektura i uczenie sieci neuronowej

Wyselekcjonowane cechy spektralne (łącznie 93 cechy wejściowe) podawane są na wejście sztucznej sieci neuronowej o architekturze zoptymalizowanej pod kątem platform TinyML:

- **Warstwa wejściowa:** 93 cechy (składowe częstotliwościowe FFT dla osi X, Y, Z).
- **Warstwa gęsta 1 (Dense):** 64 neurony z funkcją aktywacji ReLU.
- **Warstwa Dropout:** Współczynnik 0.2. Wprowadza losowe wyłączenie 20% neuronów podczas treningu, co zapobiega przeuczeniu i zmusza sieć do generalizacji.
- **Warstwa gęsta 2 (Dense):** 32 neurony z funkcją aktywacji ReLU.
- **Warstwa wyjściowa:** 5 neuronów z funkcją aktywacji Softmax, reprezentujących rozkład prawdopodobieństwa dla 5 docelowych klas.

Proces uczenia trwał niecałe 100 cykli (epochs), przy współczynniku uczenia (Learning Rate) wynoszącym 0.0005 oraz rozmiarze paczki danych (Batch Size) równym 32.



Rys. 8 Struktura zaprojektowanej sieci neuronowej

7. Opis wykonanych testów

Rozdział ten zawiera opis matematyczny oraz testowanie systemu w warunkach rzeczywistych, uzupełnione o rejestr wykrytych błędów i wprowadzonych poprawek programistycznych.

7.1. Ewaluacja statystyczna modelu (Edge Impulse)

Podczas testów na ukrytym zbiorze danych (Test Data), model sztucznej sieci neuronowej osiągnął globalną dokładność na poziomie **97.62%**. Wynik ten zweryfikowano za pomocą metryk statystycznych, wskazujących na stabilność klasyfikatora przed wdrożeniem na mikrokontroler:

- **Area under ROC Curve (1.00):** Wartość ta wskazuje na dobrą zdolność separacji klas w wykorzystanym zbiorze testowym.
- **Weighted average Precision (0.98):** Precyzja na poziomie 98% oznacza, że jeśli model zaklasyfikował dany ruch jako np. „pompki”, istnieje aż 98% szans, że użytkownik rzeczywiście wykonywał to ćwiczenie, co minimalizuje liczbę błędów typu *False Positive*.
- **Weighted average Recall (0.98):** Czułość na poziomie 98% dowodzi, że model wykrywa 98% wszystkich faktycznie wykonanych ruchów z danej klasy, co oznacza niską liczbę pominieć (*False Negatives*).
- **Weighted average F1-score (0.98):** Średnia harmoniczna precyzji i czułości, potwierdzająca dobrą stabilność statystyczną klasyfikatora na danych, których sieć nigdy wcześniej nie przetwarzała.

Results

Model version: ② Unoptimized (float32) ▾

% ACCURACY
97.62%

Metrics for Classifier

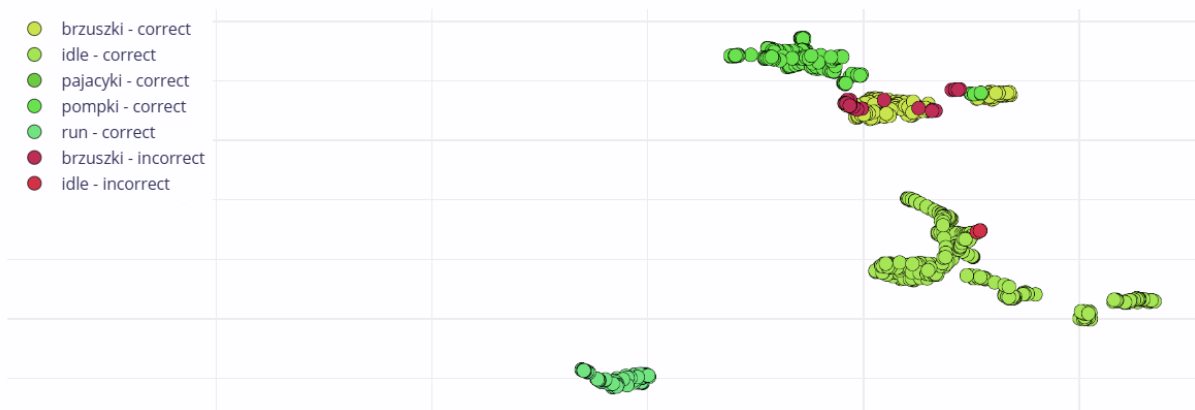


METRIC	VALUE
Area under ROC Curve ②	1.00
Weighted average Precision ②	0.98
Weighted average Recall ②	0.98
Weighted average F1 score ②	0.98

Confusion matrix

	BRZUSZKI	IDLE	PAJACYKI	POMPKI	RUN	UNCERTAIN
BRZUSZKI	88.8%	1.0%	0%	6.8%	1.0%	2.4%
IDLE	0.3%	99.5%	0%	0%	0%	0.3%
PAJACYKI	0%	0%	100%	0%	0%	0%
POMPKI	0%	0%	0%	100%	0%	0%
RUN	0%	0%	0%	0%	100%	0%
F1 SCORE	0.94	0.99	1.00	0.96	0.99	

Feature explorer ②



Rys. 9 Wyniki testowania modelu na platformie Edge Impulse

7.2. Walidacja empiryczna i przypadki brzegowe (Edge Cases)

Po optymalizacji parametrów sieci neuronowej oraz programowej inferencji, finalna wersja oprogramowania została wgrana na fizyczną platformę XIAO nRF54L15 w celu przeprowadzenia testów na żywo w warunkach rzeczywistego treningu. Urządzenie wykazało dobrą płynność działania, jednak testy na żywo ujawniły fizyczne ograniczenia logiki progowej i przetwarzania sygnału:

1. **Niejednorodne zliczanie aktywności ciągłych:** Wstępne testy zliczania biegu generowały dyskretnie zdarzenia komunikacyjne (logowanie jako pojedyncze zdarzenia run: 1, run: 2 zamiast pomiaru czasu).

2. **Pominięcia przy bardzo szybkich powtórzeniach następnych:** W przypadku wykonania serii ćwiczeń w skrajnie szybkim tempie, system pomija niektóre ruchy. Zjawisko to jest konsekwencją działania algorytmu ochronnego `cooldown_timer`, który celowo wycisza analizę na 600 ms po wykryciu powtórzenia, aby eliminować szumy fizyczne.
3. **Fałszywe zliczanie powtórzeń ("nadliczbowe pompki i brzuszki") podczas długiej statycznej pauzy:** Podczas dłuższego utrzymywania pozycji podporu przodem (plank / faza szczytowa pompki), system potrafi po kilku sekundach doliczyć fałszywe powtórzenie ćwiczenia (w tym brzuszki, mimo braku ich wykonywania). Zmęczenie mięśni powoduje drżenie rąk, a sygnał ten, przetworzony przez algorytm okna przesuwne, potrafi chwilowo zniekształcić widmo FFT, powodując oscylacje wskaźnika pewności sieci (*Confidence*) dokładnie na granicy progów decyzyjnych (90% i 40%), co maszyna stanowa błędnie interpretuje jako wykonanie kolejnego, dynamicznego powtórzenia.

7.3. Rejestr usterek i modyfikacji kodu

Poniższa tabela przedstawia historię błędów wykrytych podczas testów funkcjonalnych oraz sposób ich rozwiązania.

Kod usterki	Data	Autor	Opis usterki	Stan
ERR_0 1	9.06.2026	M. Skrobis z	Przy ponownym połączeniu BLE urządzenie pamięta stany z poprzednich sesji, przez co licznik nagle startuje od wysokich wartości (np. pompki: 22).	Rozwiązane: Wprowadzono asynchroniczne flagi czyszczące rejestry oraz zaimplementowano procedurę całkowitego resetu zmiennych sesyjnych wyzwalaną komendą 1 z aplikacji mobilnej.

Kod usterki	Data	Autor	Opis usterki	Stan
ERR_02	9.06.2026	M. Skrobisz	Wolne wykonywanie powtórzeń (faza izometryczna) powoduje podwójne zliczanie pojedynczego ruchu ze względu na mikro-przerwy w ruchu.	Rozwiązane: Zaimplementowano programowy <i>debouncing</i> w postaci timera stygnięcia (<code>cooldown_timer = 3</code>), blokujący analizę stanów na 600 ms po zaliczeniu ruchu.
ERR_03	9.06.2026	M. Skrobisz	Przenikanie się klas (<i>cross-talk</i>): podczas wykonywania pompek system losowo dolicza powtórzenia brzusków i odwrotnie, ze względu na zbliżone składowe widmowe FFT.	Rozwiązane: Podniesiono próg aktywacji maszyny stanów <code>START_THRESHOLD</code> z 0.85 na 0.90, wymagając od sieci neuronowej wyższej pewności przed zmianą stanu.
ERR_04	10.06.2026	M. Skrobisz	Aktywność ciągła (bieg/trucht) jest	Rozwiązane: Przebudowano pętlę główną <code>main.cpp</code> , wprowadzając

Kod usterki	Data	Autor	Opis usterki	Stan
			raportowana w postaci dyskretnych impulsów licznikowych (run: 1, run: 2), co uniemożliwia ocenę czasu trwania.	rejestrację czasu systemowego Zephyr RTOS (k_uptime_get_32()). System mierzy teraz czas trwania stanu i wysyła łączny wynik w sekundach (bieg: X s).

8. Podręcznik użytkownika

Niniejszy rozdział opisuje procedurę konfiguracji środowiska programistycznego, budowania projektu oraz programowania pamięci flash mikrokontrolera Seeed Studio XIAO nRF54L15 Sense.

8.1. Konfiguracja środowiska deweloperskiego

W celu skompilowania oprogramowania układowego, użytkownik musi przygotować środowisko programistyczne oparte na ekosystemie Nordic Semiconductor:

1. Zainstalować środowisko **Visual Studio Code** wraz z oficjalnym rozszerzeniem **nRF Connect for VS Code extension pack**.
2. Za pomocą panelu powitalnego *NRF CONNECT: WELCOME* wybrać opcję *Manage SDKs* oraz *Manage toolchains*.
3. Zainstalować i aktywować oficjalną wersję **nRF Connect SDK v3.0.1** oraz odpowiadający jej zestaw narzędzi kompilacji **nRF Connect SDK Toolchain v3.0.1**.

8.2. Prototypowanie i budowanie projektu (Build Configuration)

Po zaimportowaniu kodu źródłowego i bibliotek Edge Impulse (ei_model), należy poprawnie skonfigurować proces kompilacji:

1. W panelu *APPLICATIONS* wybrać opcję *Add build configuration* dla projektu *MultiSport_Tracker_TinyML*.
2. Jako docelową architekturę sprzętową (*Board target*) wybrać z listy oficjalną definicję dla układu aplikacyjnego: **xiao_nrf54l15/nrf54l15/cpuapp**.
3. Zatwierdzić konfigurację i uruchomić proces budowania systemu operacyjnego Zephyr oraz źródeł C++ klikając przycisk *Build*.

8.3. Procedura wgrywania oprogramowania (Flashing via Terminal)

Ze względu na to, że układ nRF54L15 oparty na rdzeniu ARM Cortex-M33 jest świeżą architekturą na rynku mikrokontrolerów, bezpośrednio wgrywanie oprogramowania za pomocą interfejsu graficznego (GUI) wbudowanego w rozszerzenie nRF Connect potrafi zgłosić błąd weryfikacji rejestrów IDR lub awarię debugera.

W celu poprawnego zaprogramowania urządzenia należy skorzystać z zewnętrznego narzędzia deweloperskiego **PyOCD** z poziomu wbudowanego terminala systemu operacyjnego:

1. Upewnić się, że w systemie zainstalowane jest środowisko Python oraz pakiet PyOCD (`pip install pyocd`).
2. Otworzyć terminal w głównym katalogu projektu.
3. Wywołać bezpieczne, niskopoziomowe flashowanie połączonego obrazu binarnego `merged.hex` za pomocą komendy wskazującej jawnie docelowy target mikrokontrolera: `pyocd flash build/merged.hex -t nrf54l`

4. Po pomyślnym zakończeniu przesyłania danych, mikrokontroler zresetuje się automatycznie i rozpocznie nadawanie rozgłoszeń BLE pod nazwą XIAO_FIT_TRACKER.

9. Metodologia rozwoju i utrzymania systemu

Rozdział ten definiuje wytyczne dotyczące długofalowego utrzymania oprogramowania, cyklu życia produktu oraz potencjalnych ścieżek rozwoju technologicznego prototypu.

9.1. Ścieżki optymalizacji algorytmów TinyML

Obecna wersja systemu stanowi w pełni funkcjonalny prototyp (Proof of Concept), który wskazał na stabilność założeń. W celu transformacji projektu w produkt komercyjny, metodologia rozwoju zakłada realizację trzech usprawnień programistycznych:

- **Rozbudowa bazy danych (Dataset):** Zebrany zbiór (26 minut i 14 sekund) powinien zostać rozszerzony do wielogodzinnych nagrań realizowanych przez zróżnicowaną grupę testową, co zapobiegnie przeuczeniu modelu na specyficzną technikę wykonywania ćwiczeń jednego użytkownika.
- **Optymalizacja okna przesuwnego:** Problem pomijania skrajnie szybkich ruchów można zniwelować poprzez skrócenie rozmiaru okna do 2000 ms i kroku do 100 ms. Wiąże się to jednak z koniecznością głębokiej optymalizacji pętli C++, aby układ podołał inferencji o częstotliwości 10 Hz.
- **Izolacja stanów statycznych:** W celu eliminacji fałszywych powtórzeń wywoływanych m. in. drżeniem zmęczonych mięśni podczas podporu (plank), należy wdrożyć programową bramkę wariancji sygnału w bloku DSP lub wprowadzić do sieci neuronowej dedykowaną klasę statyczną plank/hold.

9.2. Projektowanie mechaniczne i konserwacja sprzętu

Długofalowe utrzymanie urządzenia pracującego w trudnych warunkach środowiskowych (wilgoć, pot, ciągłe wibracje dynamiczne) wymaga wdrożenia odpowiedniej metodologii utrzymania warstwy fizycznej:

- **Dedykowana obudowa przemysłowa:** Kolejnym krokiem rozwojowym jest zastąpienie uniwersalnego pudełka plastikowego dedykowaną obudową zaprojektowaną w środowisku CAD i wytworzoną metodą druku 3D (SLA/FDM) z elastycznego materiału TPU. Obudowa musi posiadać zintegrowane kanały prowadzące dla podłużnego powerbanka oraz uszczelnienie silikonowe, zapewniające stopień ochrony na poziomie co najmniej **IP65** (całkowita ochrona przed potem i wilgocią).
- **Tłumienie drgań wtórnych:** Wewnętrzne usztywnienie za pomocą pianki musi być okresowo kontrolowane. Zużycie mechaniczne wkładki amortyzującej może prowadzić do powstawania luzów, co skutkuje

rejestracją drgań własnych obudowy przez akcelerometr i drastycznym spadkiem pewności predykcji sieci neuronowej.

9.3. Zarządzanie cyklem życia oprogramowania i aktualizacje

Wdrożenie systemu na rynek wiąże się z koniecznością aktualizacji sieci neuronowej bez konieczności fizycznego podłączania urządzenia do komputera deweloperskiego przez port USB-C.

System operacyjny Zephyr RTOS umożliwia wdrożenie zarządzania firmwarem za pomocą wbudowanego bootloadera **MCUboot**. Przyszłe wersje systemu powinny zostać rozwinięte o obsługę profilu **DFU over BLE** (Device Firmware Update). Pozwoli to na bezprzewodowe i bezpieczne wgrywanie zaktualizowanych wag sieci neuronowej bezpośrednio z poziomu aplikacji na smartfonie, zapewniając łatwe utrzymanie systemu u klienta końcowego.